ExpenseIQ

Group size / Theme	3 members / Expense Tracker
Members	Badreddine El Habra (164174) • Jihane Erraji (146126) • Reda Nadir (159229)
GitHub usernames	BEH-77; erraji-jihane; xizvk
Repository	https://github.com/BEH-77/ExpenseIQ
GitHub Pages	https://beh-77.github.io/ExpenseIQ/

1. Project & Role Distribution

ExpenseIQ is a lightweight Python command-line expense tracker with local JSON persistence. The two-member group divided program logic into two testable areas. **Badreddine** developed expense creation, listing, editing, deletion, validation, file storage and CLI behavior. **Jihane** developed analytics functions for totals, category breakdowns, top spending category and spending summaries. The repository also contains a static GitHub Pages information site.

2. Branches, Commits & Contributions

Badreddine: `feature/expense-management` (PRs #1, #6), with commits `b51a1bd` for core functionality, `f2d2ee7` for tests, and `6262d67`, `b916076`, `1610aca` for the CI demonstration. He also worked on `create_conflict` (PR #5), `feature/github-pages` (PR #7), and the later correction PR #8.

Jihane: `jihane_softeng` (PRs #2 and #3), including `4bac88b` for analytics, `30c9aa7` for 10 analytics tests, and `9013369` for a requested README adjustment. She also worked on `jihane_merge_conflict` (PR #4). GitHub records approvals from both members.

Reda: feature/storage-cli-updates (PR #10), including commit 260d5b3 for storage persistence helper logic and unit tests, and feature/conflict-branch-reda (PR #11) for controlled README conflict resolution. GitHub records approvals from all group members.



Figure 1. Expense-management commits recorded on the feature branch.

3. Pull Requests & Code Review

The repository contains merged PRs #1-#8. Jihane approved expense management PRs #1 and #6 and the website PR #7; Badreddine approved Jihane's PRs #2, #3 and #4. In PR #3, Badreddine commented "change something", Jihane pushed commit `9013369` to the same branch, and Badreddine subsequently approved. This documents a comment-correction-review sequence; the publicly visible record does **not** establish that the formal GitHub "Request changes" state was selected. Additionally, Reda authored PR #10 (feature/storage-cli-updates) for storage persistence helper logic and unit tests, and PR #11 (feature/conflict-branch-reda) for controlled merge conflict resolution. Peer reviews were conducted prior to merging into main, satisfying the repository's code review requirements across all three members.

4. Merge Conflict

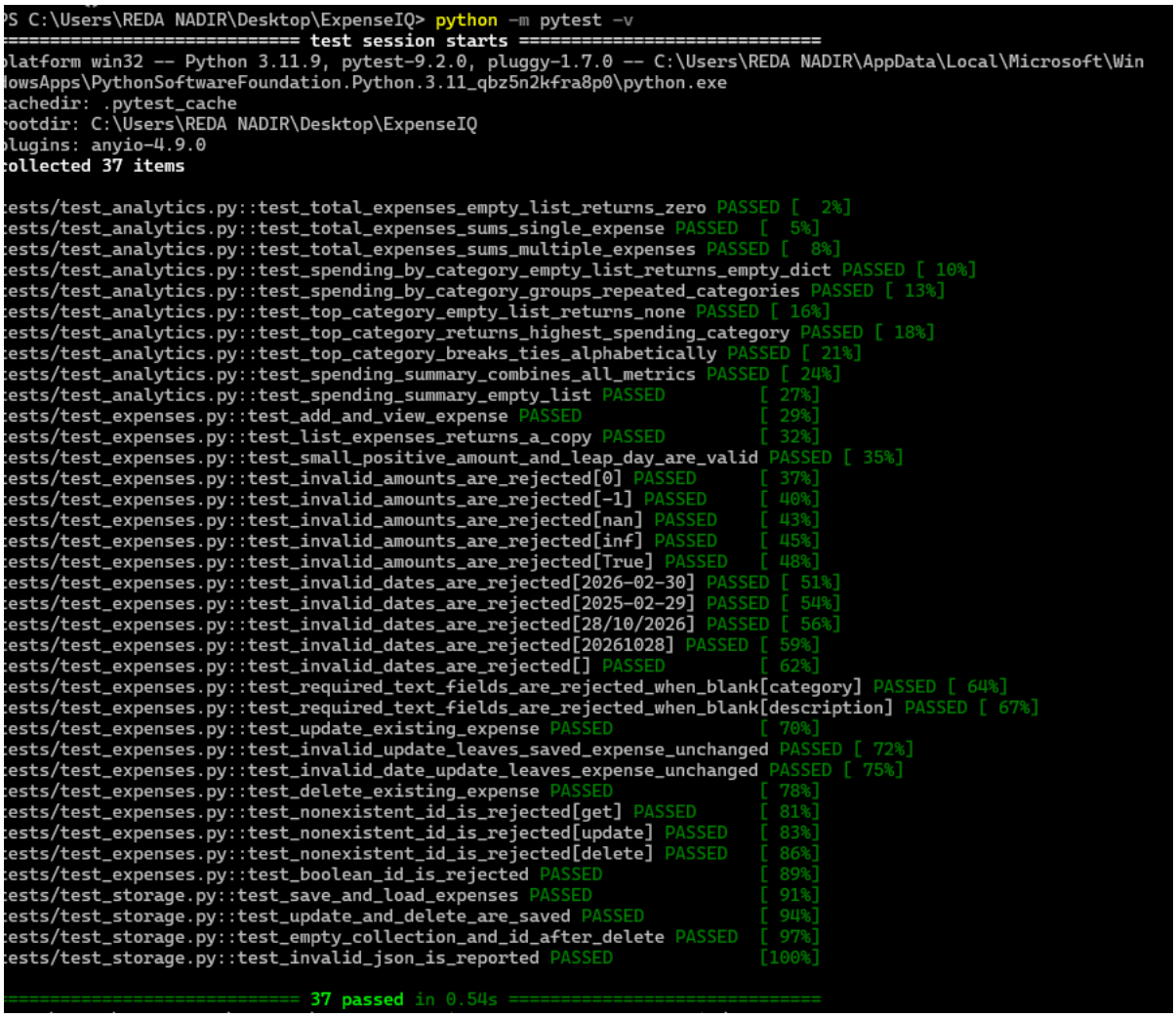
The group prepared competing README edits in branches `jihane_merge_conflict` (PR #4) and `create_conflict` (PR #5). After one change reached main, Badreddine merged main into the other branch and Git reported `CONFLICT (content): Merge conflict in README.md`. The local branch version was "Current" (HEAD), and the incoming version came from main. The conflict was resolved and committed as `a0fc1ff` ("fix conflict"). The current README retains `Project status: Development in progress.`



Figure 2. Git reports the README merge conflict, followed by a recorded resolution commit.

5. Unit Tests

The repository contains `tests/test_expenses.py`, `tests/test_storage.py`, and `tests/test_analytics.py`. Expense tests cover CRUD and invalid values; storage tests exercise JSON saving/loading and error paths; Jihane's analytics commit explicitly records 10 tests for empty input, single items, category aggregation, repeated categories and tie-breaking. The PR #8 description reports **37 locally passing tests**; this is a contributor-reported result, independently supported by passing GitHub Actions checks, not a new local execution performed for this report. Reda expanded the test suite in tests/test_storage.py with 4 dedicated storage persistence unit tests covering saving, loading, collection updating, empty collection handling, post-deletion ID tracking, and corrupted JSON error handling, bringing the total suite to 37 passing unit tests verified locally and via CI.



6. Continuous Integration

The `.github/workflows/tests.yml` workflow uses GitHub Actions to run pytest on pushes and pull requests. Its documented role is to prepare Python and dependencies on an Ubuntu runner and execute the project's test suite. GitHub Actions shows deliberately failed CI commits `b916076` ("ci: demonstrate failed pytest workflow") and a subsequent restoration `1610aca` ("ci: restore passing pytest workflow"). The precise failing command is not established from the supplied screenshot alone; the version-control history verifies the controlled fail-and-restore sequence.

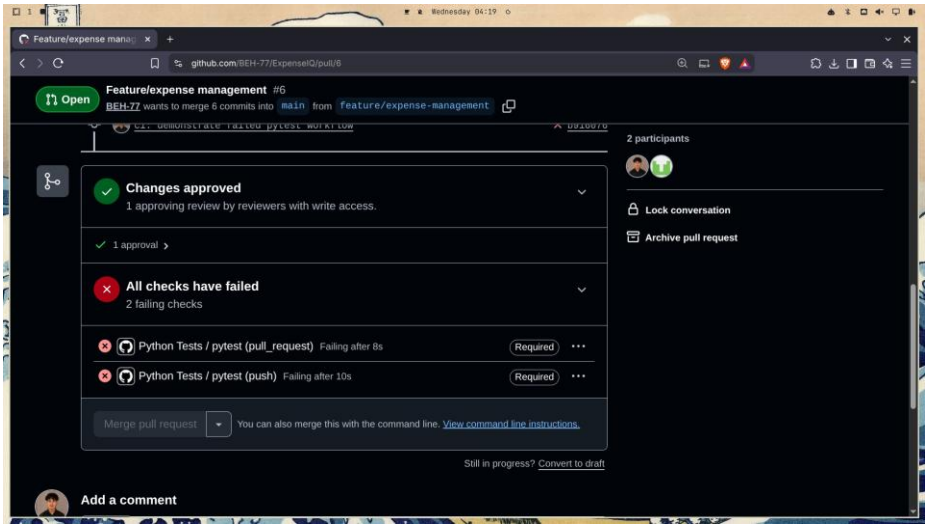


Figure 3. Required push and pull-request pytest checks failed, disabling the merge button.

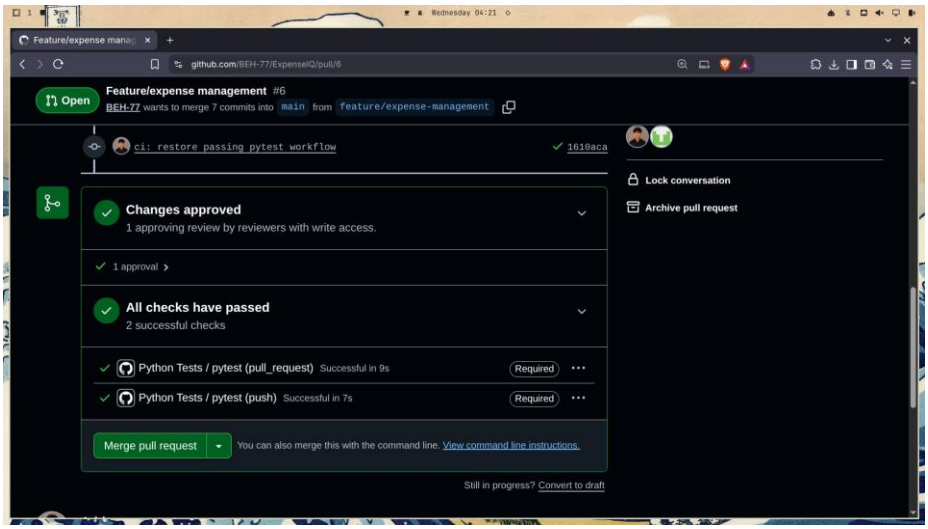


Figure 4. Both required pytest checks subsequently passed and merging became available.

7. Ruleset & Protected main

The PR screenshots show an approval from a reviewer with write access and two checks marked "Required": `Python Tests / pytest (pull_request)` and `Python Tests / pytest (push)`. The failed-check screenshot shows the merge control disabled; the successful-check screenshot shows it enabled. This demonstrates review and CI gating. Deletion and force-push restrictions cannot be independently confirmed from the supplied public evidence.

8. Continuous Deployment

The static website is stored under `docs/`, and the repository README identifies `main` / `docs` as its Pages publication source. PR #7 introduced the site; a subsequent PR #8 corrected the displayed team member name and documentation. The Actions history records successful Pages deployments #2 and #3 on October 28, 2026. The live page now displays "Badr & Jihane Erraji" and the CLI instructions, confirming the published update.

9. Contribution Summary

Member	Features / branches	PRs and review evidence	Tests
Badreddine El Habra	Expense management, JSON/CLI, CI and website; feature/expense-management, create_conflict, feature/github-pages	Authored #1, #5, #6, #7, #8; reviewed Jihane's #2, #3 and #4	tests/test_expenses.py and tests/test_storage.py
Jihane Erraji	Analytics and README changes; jihane_softeng, jihane_merge_conflict	Authored #2, #3 and #4; reviewed #1, #5, #6 and #7	10 analytics tests in tests/test_analytics.py
Reda Nadir	Storage persistence, CLI helper logic, conflict resolution; `feature/storage-cli-updates`	Authored #10, #11; completed review cycles	4 storage unit tests in `tests/test_storage.py` (37 total)

10. Fair Use of AI

AI tools were used to assist with writing project code, tests and this report. Prompts concerned expense-tracker features, pytest coverage, workflow configuration, documentation, and summarizing verifiable repository evidence. Generated material was compared with repository files, commit and Pull Request history, recorded CI results, and the live project page. The team remains responsible for understanding and checking all submitted code and explanations. The use of AI by each member individually was not separately specified.

11. Sources & Reflection

Sources: Assignment 2 instructions; GitHub repository README and project code; Pull Requests #1-#8; GitHub Actions run history; public GitHub Pages site; supplied terminal and PR screenshots. Relevant URLs: github.com/BEH-77/ExpenseIQ ; github.com/BEH-77/ExpenseIQ/pulls ; github.com/BEH-77/ExpenseIQ/actions ; beh-77.github.io/ExpenseIQ/.

The most challenging visible workflow involved reconciling a README merge conflict and managing intentionally failing CI checks. Git first refused another merge while README.md contained unresolved paths, illustrating why conflicts must be resolved and staged before committing. The required pytest checks then blocked the Pull Request until a corrected workflow was pushed and both checks passed. If collaboration or automation fails again, the group should inspect the current branch, unmerged files, Pull Request review state, and failing Actions logs before attempting another merge.

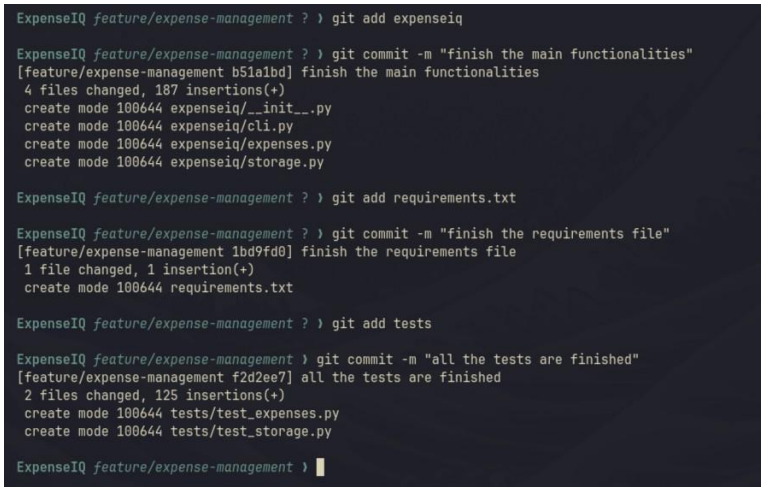


Figure 5. Example early expense-management commits on the feature branch.

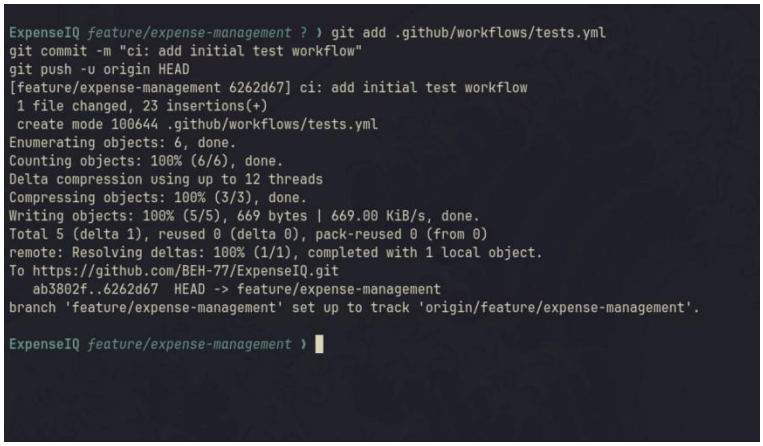


Figure 6. Initial GitHub Actions workflow committed and pushed.

Evidence note: Figures 3-4 are snapshots of different CI states, not screenshots of the complete test logs. The public Actions history provides the additional run-level verification cited in the report.